4. The main advantage of run-time polymorphism is flexibility. The main disadvantage is loss of execution speed.

11.1 EXERCISES

2.
```
#include <iostream>
using namespace std;

template <class X> X min(X a, X b)
{
    if(a<=b)
        return a;
    else
        return b;
}

int main()
{
    cout << min(12.2, 2.0);
    cout << endl;
    cout << min(3, 4);
    cout << endl;
    cout << min('c', 'a');

    return 0;
}

/*
   This min(X, X) function might cause ambiguity
   because of STL function min(const _Tp&, const _Tp&).
   Try a different name.
*/
```

3.
```
#include <iostream>
#include <cstring>
using namespace std;

template <class X> int find(X object, X *list, int size)
{
    int i;

    for(i=0; i<size; i++)
        if(object == list[i])
            return i;
    return -1;
}

int main()
{
    int a[] = {1, 2, 3, 4};
    char *c = "this is a test";
```

```
    double d[] = {1.1, 2.2, 3.3};

    cout << find(3, a, 4);
    cout << endl;
    cout << find('a', c, (int) strlen(c));
    cout << endl;
    cout << find(0.0, d, 3);

    return 0;
}
```

4. Generic functions are valuable because they allow you to define a general algorithm that can be applied to various types of data. (That is, specific versions of the algorithm need not be explicitly created by you.) Generic functions further help implement the concept of "one interface, multiple methods," which is a common theme in C++ programming.

11.2 EXERCISES

2.
```
// Create a generic queue.
#include <iostream>
using namespace std;

#define SIZE 100

template <class Qtype> class q_type
{
    Qtype queue[SIZE]; // holds the queue
    int head, tail; // indices of head and tail
public:
    q_type() { head = tail = 0; }
    void q(Qtype num); // store
    Qtype deq(); // retrieve
};

// Put value on queue.
template <class Qtype> void q_type<Qtype>::q(Qtype num)
{
    if(tail+1==head || (tail+1==SIZE && !head))
    {
        cout << "Queue is full.\n";
        return;
    }
    tail++;
    if(tail==SIZE)
        tail = 0; // cycle around
    queue[tail] = num;
}

// Remove value from queue.
template <class Qtype> Qtype q_type<Qtype>::deq()
{
```

```
    if(head == tail)
    {
        cout << "Queue is empty.\n";
        return 0; // or some other error indicator
    }
    head++;
    if(head==SIZE)
        head = 0; // cycle around
    return queue[head];
}

int main()
{
    q_type<int> q1;
    q_type<char> q2;
    int i;

    for(i=1; i<=10; i++)
    {
        q1.q(i);
        q2.q(i-1+'A');
    }

    for(i=1; i<=10; i++)
    {
        cout << "Dequeue 1: " << q1.deq() << '\n';
        cout << "Dequeue 2: " << q2.deq() << '\n';
    }

    return 0;
}
```

3.
```
#include <iostream>
using namespace std;

template <class X> class input
{
    X data;
public:
    input(char *s, X min, X max);
    // ...
};

template <class X>
input<X>::input(char *s, X min, X max)
{
    do
    {
        cout << s << ": ";
        cin >> data;
    }
```

```
    while(data < min || data > max);
}

int main()
{
    input<int> i("enter int", 0, 10);
    input<char> c("enter char", 'A', 'Z');

    return 0;
}
```

11.3 EXERCISES

2. The **throw** is called before execution passes through a **try** block.

3. A character exception is thrown, but the **catch** statement will handle only a character pointer. (That is, there is no corresponding **catch** statement to handle the character exception.)

4. If an exception is thrown for which there is no corresponding **catch**, **terminate()** is called and abnormal program termination might occur.

11.4 EXERCISES

2. There is no corresponding **catch** statement for the **throw**.

3. One way to fix the problem is to create a **catch(int)** handler. Another way to fix it is to catch all exceptions with a **catch(...)** handler.

4. **catch(...)** catches all exceptions.

5. ```
   #include <iostream>
   #include <cstdlib>
   using namespace std;

   double divide(double a, double b)
   {
       try
       {
           if(!b)
               throw(b);
       }
       catch(double)
       {
           cout << "Cannot divide by zero.\n";
           exit(1);
       }
       return a/b;
   }

   int main()
   {
   ```
   ```

```
    cout << divide(10.0, 2.5) << endl;
    cout << divide(10.0, 0.0);

    return 0;
}
```

11.5 EXERCISES

1. By default, **new** throws an exception when an allocation error occurs. The **nothrow** version of **new** returns a null pointer if memory cannot be allocated.

2. ```
   p = new(nothrow) int;

   if(!p)
   {
       cout << "Allocation error.\n";
       // ...
   }

   try
   {
       p = new int;
   }
   catch(bad_alloc ba)
   {
       cout << "Allocation error.\n";
       // ...
   }
   ```

MASTERY SKILLS CHECK: Chapter 11

1. ```
   #include <iostream>
   #include <cstring>
   using namespace std;

   // A generic mode-fining function.
   template <class X> X mode(X *data, int size)
   {
       register int t, w;
       X md, oldmd;
       int count, oldcount;

       oldmd = 0;
       oldcount = 0;
       for(t=0; t<size; t++)
       {
           md = data[t];
           count = 1;
           for(w = t+1; w<size; w++)
           {
   ```

```
            if(md==data[w])
                count++;
            if(count > oldcount)
            {
                oldmd = md;
                oldcount = count;
            }
        }
    }
    return oldmd;
}

int main()
{
    int i[] = {1, 2, 3, 4, 2, 3, 2, 2, 1, 5};
    char *p = "this is a test";

    cout << "mode of i: " << mode(i, 10) << endl;
    cout << "mode of p: " << mode(p, (int)strlen(p)) << endl;

    return 0;
}
```

2.
```
#include <iostream>
using namespace std;

template <class X> X sum(X *data, int size)
{
    int i;
    X result = 0;

    for(i=0; i<size; i++)
        result += data[i];

    return result;
}

int main()
{
    int i[] = {1, 2, 3, 4};
    double d[] = {1.1, 2.2, 3.3, 4.4};

    cout << sum(i, 4) << endl;
    cout << sum(d, 4) << endl;

    return 0;
}
```

3.
```
#include <iostream>
using namespace std;
```

```
// A generic bubble sort.
template <class X> void bubble(X *data, int size)
{
    register int a, b;
    X t;

    for(a=1; a<size; a++)
    {
        for(b=size-1; b>=a; b--)
        {
            if(data[b-1]>data[b])
            {
                t = data[b-1];
                data[b-1] = data[b];
                data[b] = t;
            }
        }
    }
}

int main()
{
    int i[] = {3, 2, 5, 6, 1, 8, 9, 3, 6, 9};
    double d[] = {1.2, 5.5, 2.2, 3.3};
    int j;

    bubble(i, 10); // sort ints
    bubble(d, 4);  // sort doubles

    for(j=0; j<10; j++)
        cout << i[j] << ' ';
    cout << endl;

    for(j=0; j<4; j++)
        cout << d[j] << ' ';
    cout << endl;

    return 0;
}
```

4.
```
/*
   This function demonstrates a generic stack that
   holds two values.
*/
#include <iostream>
using namespace std;

#define SIZE 10

// Create a generic stack class
template <class StackType> class stack
```

```
{
    StackType stck[SIZE][2]; // holds the stack
    int tos; // index of top of stack
public:
    void init() { tos = 0; }
    void push(StackType ob, StackType ob2);
    StackType pop(StackType &ob2);
};

// Push objects.
template <class StackType>
void stack<StackType>::push(StackType ob, StackType ob2)
{
    if(tos==SIZE)
    {
        cout << "Stack is full.\n";
        return;
    }
    stck[tos][0] = ob;
    stck[tos][1] = ob2;
    tos++;
}

// Pop objects.
template <class StackType>
StackType stack<StackType>::pop(StackType &ob2)
{
    if(tos==0)
    {
        cout << "Stack is empty.\n";
        return 0; // return null on empty stack
    }
    tos--;
    ob2 = stck[tos][1];
    return stck[tos][0];
}

int main()
{
    // Demonstrate character stacks.
    stack<char> s1, s2; // create two stacks
    int i;
    char ch;

    // initialize the stacks
    s1.init();
    s2.init();

    s1.push('a', 'b');
    s2.push('x', 'z');
    s1.push('b', 'd');
```

```
    s2.push('y', 'e');
    s1.push('c', 'a');
    s2.push('z', 'x');

    for(i=0; i<3; i++)
    {
        cout << "Pop s1: " << s1.pop(ch);
        cout << ' ' << ch << '\n';
    }
    for(i=0; i<3; i++)
    {
        cout << "Pop s2: " << s2.pop(ch);
        cout << ' ' << ch << '\n';
    }

    // demonstrate double stacks
    stack<double> ds1, ds2; // create two stacks
    double d;

    // initialize the stacks
    ds1.init();
    ds2.init();

    ds1.push(1.1, 2.0);
    ds2.push(2.2, 3.0);
    ds1.push(3.3, 4.0);
    ds2.push(4.5, 5.0);
    ds1.push(5.5, 6.0);
    ds2.push(6.6, 7.0);

    for(i=0; i<3; i++)
    {
        cout << "Pop ds1: " << ds1.pop(d);
        cout << ' ' << d << '\n';
    }
    for(i=0; i<3; i++)
    {
        cout << "Pop ds2: " << ds2.pop(d);
        cout << ' ' << d << '\n';
    }

    return 0;
}
```

5. The general forms of **try**, **catch**, and **throw** are shown here:

```
try
{
    // try block
    throw exp;
}
catch(type arg)
```

```
{
    // ...
}
```

6.
```
/*
   This function demonstrates a generic stack
   that includes exception handling.
*/
#include <iostream>
using namespace std;

#define SIZE 10

// Create a generic stack class
template <class StackType> class stack
{
    StackType stck[SIZE]; // holds the stack
    int tos; // index of top of stack
public:
    void init() { tos = 0; }
    void push(StackType ob);
    StackType pop();
};

// Push objects.
template <class StackType>
void stack<StackType>::push(StackType ob)
{
    try
    {
        if(tos==SIZE)
            throw SIZE;
    }
    catch(int)
    {
        cout << "Stack is full.\n";
        return;
    }
    stck[tos] = ob;
    tos++;
}

// Pop objects.
template <class StackType>
StackType stack<StackType>::pop()
{
    try
    {
        if(tos==0)
            throw 0;
    }
```

```
    catch(int)
    {
        cout << "Stack is empty.\n";
        return 0; // return null on empty stack
    }
    tos--;
    return stck[tos];
}

int main()
{
    // Demonstrate character stacks.
    stack<char> s1, s2; // create two stacks
    int i;
    char ch;

    // initialize the stacks
    s1.init();
    s2.init();

    s1.push('a');
    s2.push('x');
    s1.push('b');
    s2.push('y');
    s1.push('c');
    s2.push('z');

    for(i=0; i<3; i++)
        cout << "Pop s1: " << s1.pop() << '\n';
    for(i=0; i<3; i++)
        cout << "Pop s2: " << s2.pop() << '\n';

    // demonstrate double stacks
    stack<double> ds1, ds2; // create two stacks
    double d;

    // initialize the stacks
    ds1.init();
    ds2.init();

    ds1.push(1.1);
    ds2.push(2.2);
    ds1.push(3.3);
    ds2.push(4.5);
    ds1.push(5.5);
    ds2.push(6.6);

    for(i=0; i<3; i++)
        cout << "Pop ds1: " << ds1.pop() << '\n';
    for(i=0; i<3; i++)
        cout << "Pop ds2: " << ds2.pop() << '\n';
```

```
    return 0;
}
```

7. If **new** throws an exception when an allocation error occurs, you can be sure that the error will be handled one way or another-even if only by abnormal program termination. In contrast, an allocation failure that is reported by **new**, a return of a null pointer can be overlooked if you forget to check for this possibility. The trouble is that when your program attempts to use the null pointer, it might work for a while, then behave erratically, and finally crash in unpredictable (and unduplicatable) ways. This is very difficult type of bug to diagnose.

REVIEW SKILLS CHECK: Chapter 12

1. In C++, a generic function defines a general set of operations that will be applied to various types of data. It is implemented with the keyword **template**. Its general form is shown here:

```
template <class Ttype> ret_type func_name(para_list)
{
    // ...
}
```

2. In C++, a generic class defines all operations that relate to that class, but the actual data is specified as a parameter when an object of that class is created. Its general form is shown here:

```
template<class Ttype> class class_name
{
    // ...
};
```

3.
```
#include <iostream>
using namespace std;

// Return a to the b.
template <class X> X gexp(X a, X b)
{
    X i, result=1;

    for(i=0; i<b; i++)
        result *= a;

    return result;
}

int main()
{
    cout << gexp(2, 3) << endl;
    cout << gexp(10.0, 2.0);

    return 0;
}
```

4.
```cpp
#include <iostream>
#include <fstream>
using namespace std;

template<class CoordType> class coord
{
    CoordType x, y;
public:
    coord(CoordType i, CoordType j) { x = i; y = j; }
    void show() { cout << x << ", " << y << endl; }
};

int main()
{
    coord<int>o1(1, 2), o2(3, 4);

    o1.show();
    o2.show();

    coord<double> o3(0.0, 0.23), o4(10.19, 3.098);

    o3.show();
    o4.show();

    return 0;
}
```

5. **try**, **catch**, and **throw** work together like this: Put all statements that you wish to monitor for exceptions within a **try** block, if an exception occurs, throw that exception using **throw** and handle it with a corresponding **catch** statement.

6. No.

7. **terminate()** is called when an exception is thrown for which there is no corresponding **catch** statement. **unexpected()** is called when an attempt is made to throw an exception out of a function that is not specified in the function's **throw** clause.

8. **catch(...)**.

12.1 EXERCISES

1. RTTI is necessary to C++ because it is not always possible to know at compile time what type of object is being pointed to by a base class pointer of referenced by a base class reference.

2. Here is the output produced when **BaseClass** is no longer a polymorphic class.

```
Typeid of i is int
p is pointing to an object of type class BaseClass
p is pointing to an object of type class BaseClass
p is pointing to an object of type class BaseClass
```